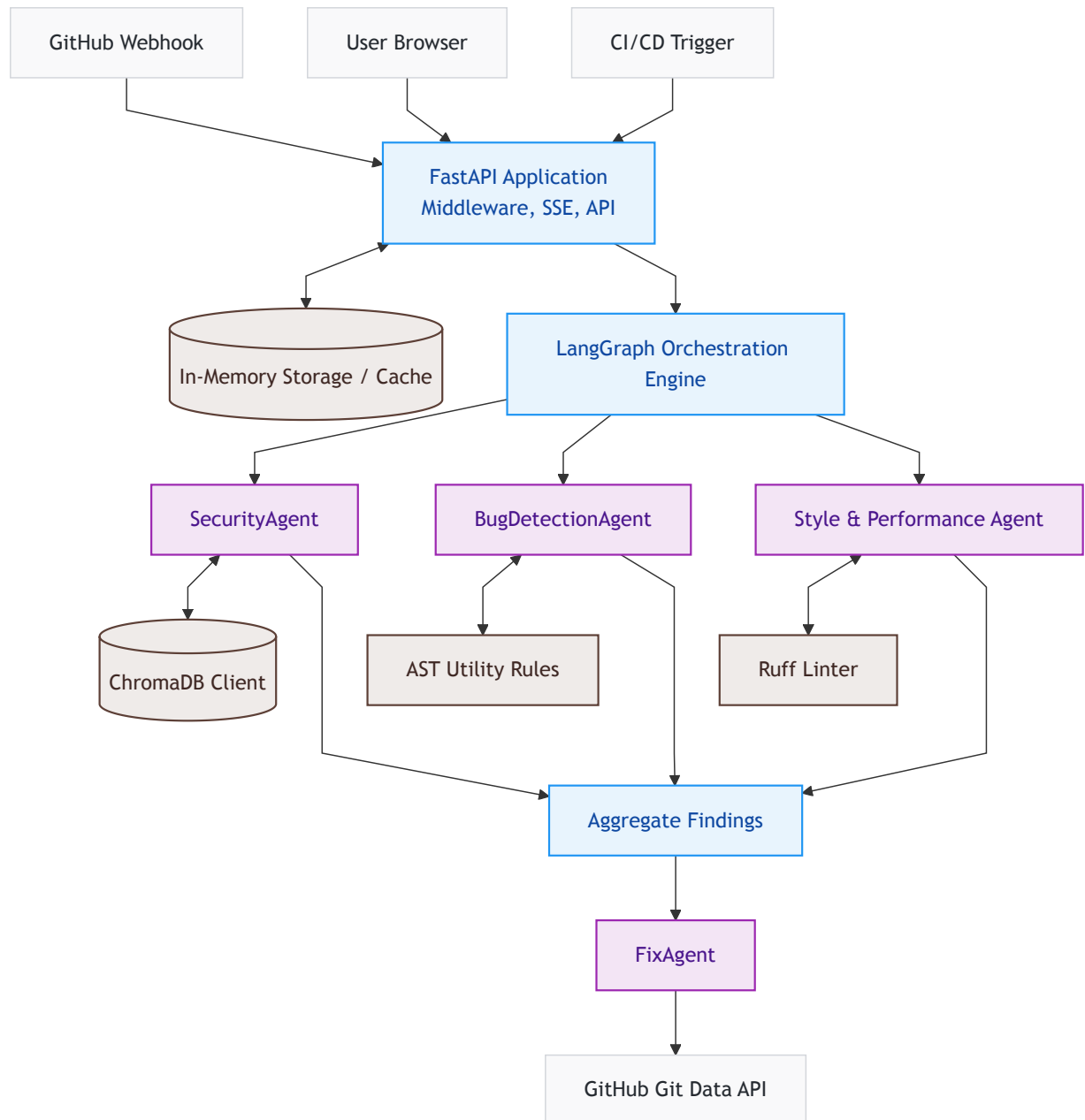


Multi-Agent LLM Based PR Review System : Architectural Blueprints

Part 1: High-Level Design (HLD)

1.1 System Architecture & Data Topology

The system uses a decoupled, event-driven, multi-agent architecture where an API layer handles intake and streaming, while an orchestration layer runs asynchronous analysis workloads via Chroma DB Retrieved Augments & localized rule-engines and a Large Language Model (LLM).



1.2 Structural Core Component Directory

Component	Use Case
FastAPI Core	Manages webhook ingestion, security validation, rate limiting, and real-time SSE streaming.
LangGraph Orchestrator	Coordinates parallel worker execution, routing logic, and multi-agent state tracking.
Static Analyzers	Executes fast AST analysis and syntax checking (e.g., <code>ruff</code>) without relying on an LLM.
Vector Index Client	Fetches relevant code snippets to provide contextually rich background data.
Virtual Git Data Service	Pushes file modifications directly to GitHub by manipulating blobs and commit trees over the API.

Part 2: Low-Level Design (LLD)

2.1 State Management Engine (`src/agents/orchestrator/state.py`)

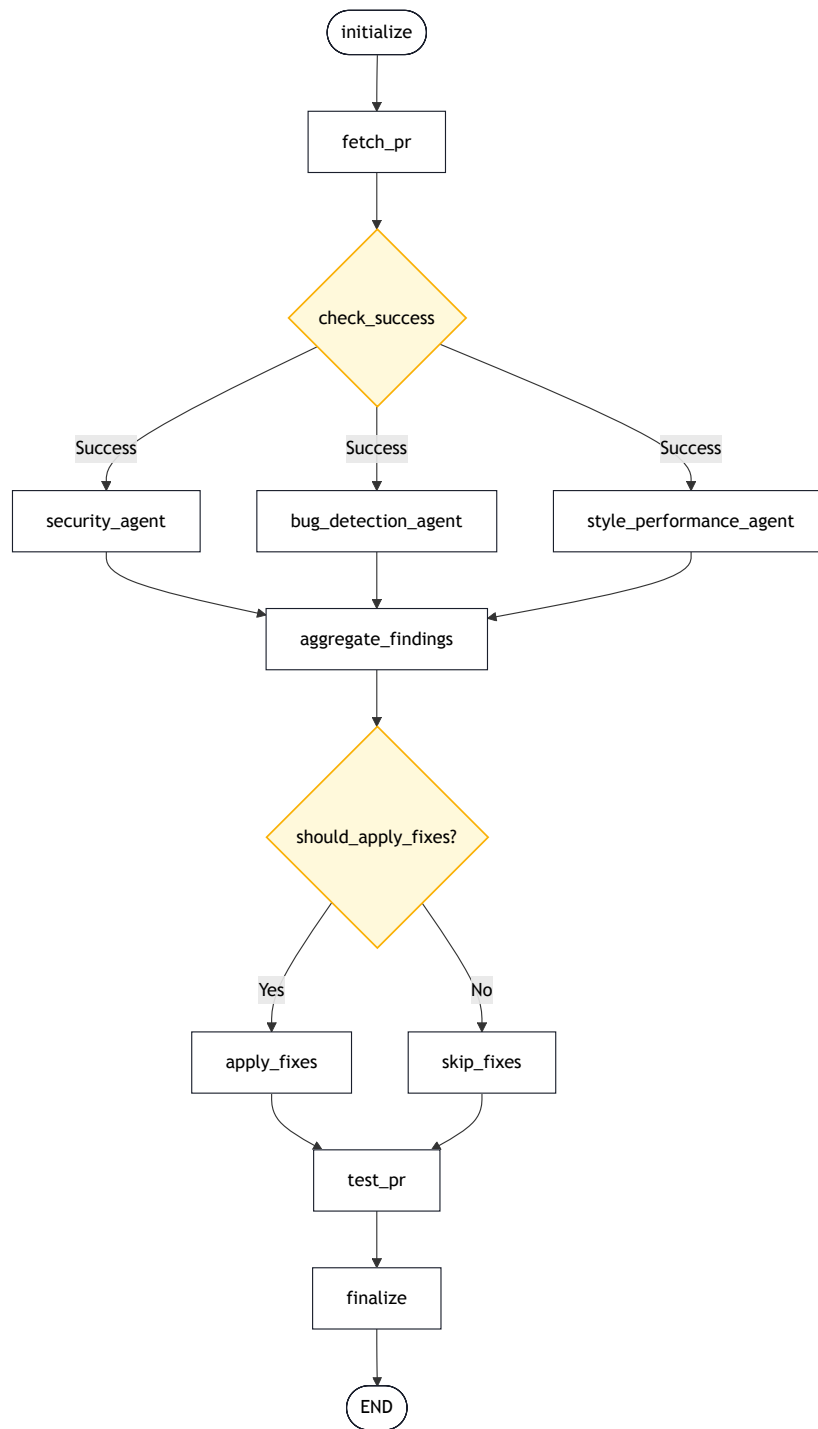
The orchestration framework uses parallel reducer methods inside `PRReviewState` maps to eliminate data-clobbering risks when multi-threaded analysis workers write back variables concurrently.

```
from typing import Annotated, TypedDict, Any
from src.models.finding import Finding

def add_findings(old_findings: list[Finding], new_findings:
list[Finding]) -> list[Finding]:
    """Appends newly discovered analytical findings from parallel
worker execution tracks."""
    return old_findings + new_findings

def add_agent_results(old_results: dict[str, Any], new_results:
dict[str, Any]) -> dict[str, Any]:
    """Merges execution meta tracking parameters (duration, success
logs) from parallel agents safely."""
    return {**old_results, **new_results}

class PRReviewState(TypedDict):
    pr_info: dict[str, Any] # PR metadata payload
    identifiers
    files: dict[str, str] # In-memory code catalog
    mapping {"file_path.py": "raw_source_code"}
    status: str # Enums tracking current
    execution progress state
    findings: Annotated[list[Finding], add_findings]
    agent_results: Annotated[dict[str, Any], add_agent_results]
    fix_results: list[dict[str, Any]]
    errors: list[str]
```

2.2 Deep Dive: Agent Internal Processing Workflows

1. Base Analysis Agent (`src/agents/base.py`)

Abstract foundational definition layer mapping target patterns. Key capabilities include:

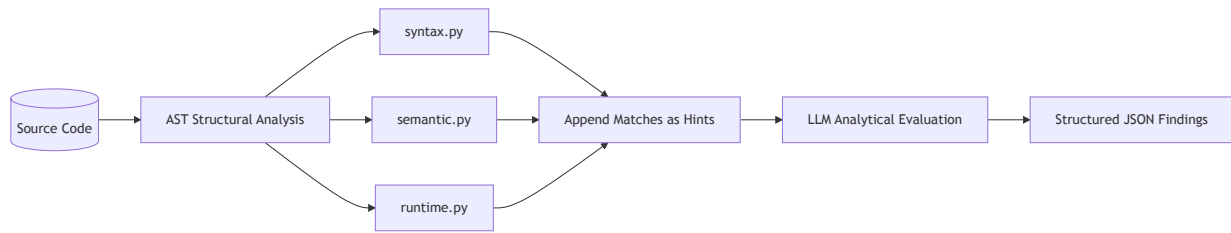
- Pre-filtering processing collections by comparing raw file scopes against configured extensions (`SOURCE_EXTENSIONS`).
- Wrapping runtime processing loops in isolated per-file `try/except` scopes. A corruption event inside a single file cannot halt the global execution queue.
- Enforcing programmatic tag tracking by embedding `agent_name` attributes into every discovered issue payload.

2. SecurityAgent

- Vector Engine Layer: Extracts code structures, clips payloads exceeding 2,000 characters, and queries ChromaDB for the top 5 matching compliance entries or CWE definitions.
- Safety Fallback: If ChromaDB encounters a network timeout, it falls back to a hardcoded 8-entry baseline reference rule map covering injection flaws and cryptographic issues.
- LLM Pipeline: Feeds the combined vector context and code structures to the `security.j2` template, locking execution parameters to `temperature=0` to ensure highly deterministic, structured JSON models.

3. BugDetectionAgent

Uses a dual-track strategy. It executes local structural inspections across target source spaces before generating an LLM context call.



4. StyleAgent

- Subprocess Execution Track: Spawns an isolated `ruff` sub-shell running the explicit command parameter `--output-format json` to instantly catch conventional linting discrepancies.
- LLM Reconciliation: Passes the raw source code along with the Ruff issue reports down to the LLM. It filters out any duplicate AI findings where the issue title matches an existing Ruff notification to prevent redundant alerts.

5. PerformanceAgent

Evaluates resource leaks, nesting complexities, and processing loops using deterministic Abstract Syntax Trees. It avoids ternary operators when computing constraints to protect against logical evaluation bugs.

6. FixAgent

Automates code correction using a strict, step-by-step pipeline:

1. Identify and Filter: Selects discovered issues matching acceptable severities (`critical`, `high`, `medium`).
2. Apply Partition Caps: Imposes a processing limit of 10 files per category block (`MAX_FIX_FILES_PER_CATEGORY`) to prevent runaway API tokens.
3. Refactor via LLM: Uses the `fix.j2` code generation template to produce corrected source code snippets based on the grouped issues.
4. Compile and Verify: Passes the generated string into `compile()` to build an internal AST tree, catching any malformed syntax before creating remote changes.
5. Direct Commit Integration: Commits valid file sets sequentially across category stages (`security` → `bug` → `style` → `performance`), using the previous stage's corrected code as the starting point for the next.

Part 3: Engineering Mechanics & Implementation Implementations

3.1 Custom AST Rule Checking Engine (`src/agents/ast_utils.py`)

This engine programmatically analyzes python files to find processing anti-patterns before making an external model call.

```

import ast
from typing import Optional, List, Dict, Any

def attach_parents(tree: ast.AST) -> ast.AST:
    """Recursively annotates all nodes in the AST with a pointer to
    their parent node."""
    for parent in ast.walk(tree):
        for child in ast.iter_child_nodes(parent):
            child.parent = parent
    return tree

def get_loop_depth(node: ast.AST) -> int:
    """Calculates the nesting depth of a given AST node within loop
    structures."""
    depth = 0
    current = getattr(node, 'parent', None)
    while current is not None:
        if isinstance(current, (ast.For, ast.While)):
            depth += 1
        current = getattr(current, 'parent', None)
    return depth

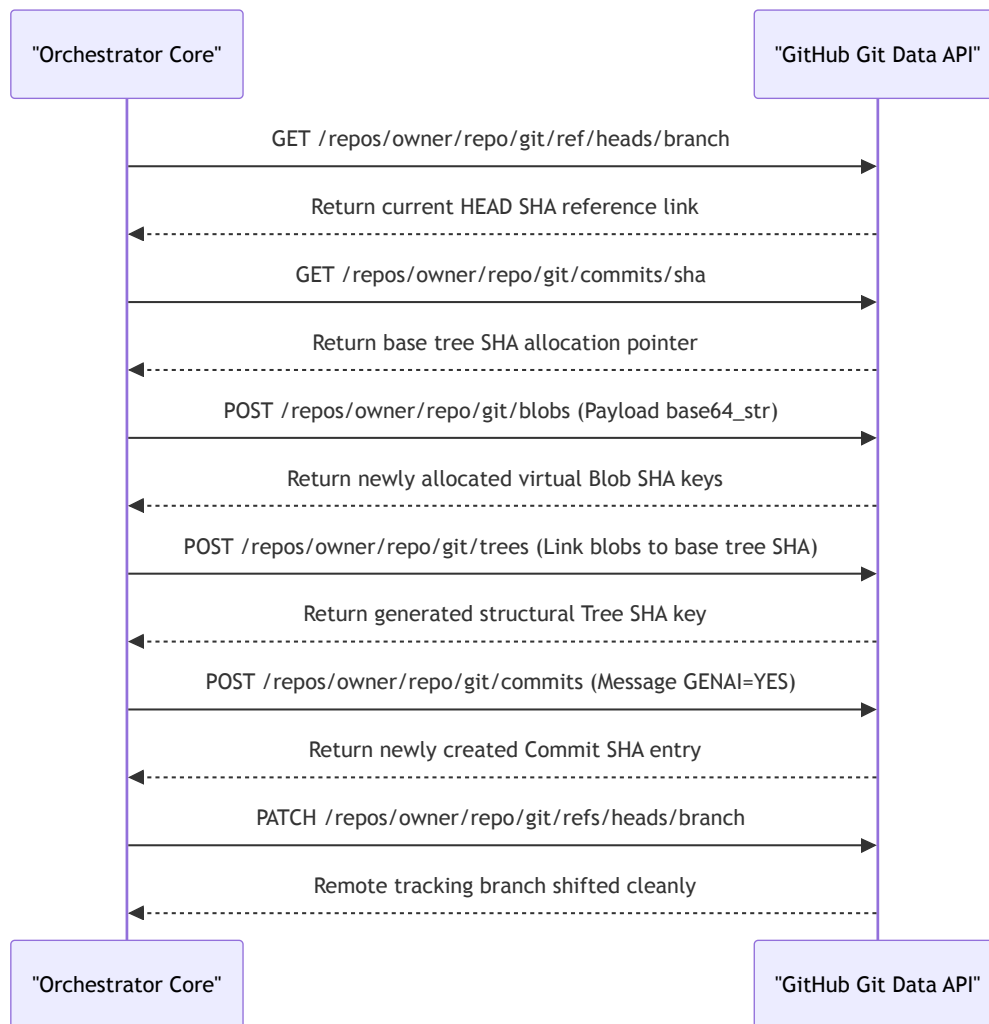
class PerformanceAnalyzer(ast.NodeVisitor):
    """Flags performance hot spots, such as string concats inside
    loops."""
    def __init__(self):
        self.hints: List[Dict[str, Any]] = []

    def visit_AugAssign(self, node: ast.AugAssign):
        if isinstance(node.op, ast.Add) and isinstance(node.target,
ast.Name):
            if get_loop_depth(node) > 0:
                # Basic string detection heuristic based on common
naming conventions
                if 'str' in node.target.id or 'text' in node.target.id:
                    self.hints.append({
                        "line": node.lineno,
                        "type": "hotspot",
                        "message": f"In-loop string concatenation
discovered on '{node.target.id}'."
                    })
            self.generic_visit(node)

```

3.2 Virtual Git Data Engine (No-Clone Branch Construction)

This sequence diagram illustrates the low-level interactions with the GitHub Git Data API to inject refactored code adjustments directly into the remote tracking branch without checking out repository files locally.



Part 4: Network Stream & Execution Infrastructure

4.1 Server-Sent Events Core Streaming Logic (`src/api/endpoints/sse.py`)

This segment handles real-time data streaming over HTTP using an on-demand async queue mapping model.

```
import asyncio
import json
from fastapi import APIRouter, Request
from fastapi.responses import StreamingResponse

router = APIRouter()
_channels: dict[str, asyncio.Queue] = {}

def get_or_create_channel(review_id: str) -> asyncio.Queue:
    """
    Returns an existing event channel or instantiates a new one.
    Crucial: Creates the channel allocation when the first publisher
    event occurs
    to prevent event loss from early connection handshakes.
    """
    if review_id not in _channels:
        _channels[review_id] = asyncio.Queue()
    return _channels[review_id]

async def event_generator(review_id: str):
    queue = get_or_create_channel(review_id)
    try:
        while True:
            try:
                # Await incoming events or trigger a ping frame every
                30 seconds
                event = await asyncio.wait_for(queue.get(),
                timeout=30.0)
                yield f"data: {json.dumps(event)}\n\n"

                # Close the stream on terminal statuses
                if event.get("status") in ["COMPLETED", "FAILED"]:
                    break
            except asyncio.TimeoutError:
                yield "data: {\"type\": \"ping\"}\n\n"
    finally:
        _channels.pop(review_id, None)

@router.get("/sse/reviews/{review_id}")
async def stream_review_events(review_id: str, request: Request):
    return StreamingResponse(event_generator(review_id),
    media_type="text/event-stream")
```

Next Steps:

Which model to use or train one ?

Dynamic Chroma DB update

Gaurdrails & Safety evaluation

System Evaluation

QA

Production Readiness